

1. Instalasi Awal

```
npx create-next-app@latest my-jwt-app
cd my-jwt-app
npm install jsonwebtoken bcryptjs
```

2. Struktur Folder

```
my-jwt-app/
├── lib/
│   └── users.js      # Simulasi database
├── middleware/
│   └── auth.js       # JWT verification
└── pages/
    └── api/
        ├── register.js # Register endpoint
        ├── login.js    # Login endpoint
        └── protected.js # Endpoint terproteksi
```

3. Simulasi Database – lib/users.js

```
let users = []

export function getUserByEmail(email) {
  return users.find((u) => u.email === email)
}

export function addUser(user) {
  users.push(user)
}
```

Untuk versi produksi, ganti dengan database seperti MongoDB/Prisma.

4. Middleware JWT – middleware/auth.js

```
import jwt from 'jsonwebtoken'

const SECRET = 'your-secret-key'
```

```
export function authenticateToken(req, res) {
  const authHeader = req.headers.authorization
  const token = authHeader?.split(' ')[1]

  if (!token) return res.status(401).json({ error: 'Token missing' })

  try {
    return jwt.verify(token, SECRET)
  } catch (err) {
    return res.status(403).json({ error: 'Invalid token' })
  }
}
```

5. API Register – pages/api/register.js

```
import bcrypt from 'bcryptjs'
import { addUser, getUserByEmail } from '@/lib/users'

export default async function handler(req, res) {
  if (req.method !== 'POST') return res.status(405).end()

  const { email, password } = req.body

  if (getUserByEmail(email)) return res.status(400).json({ error: 'User already exists' })

  const hashedPassword = await bcrypt.hash(password, 10)
  addUser({ email, password: hashedPassword })

  res.status(201).json({ message: 'User registered' })
}
```

6. API Login – pages/api/login.js

```
import jwt from 'jsonwebtoken'
import bcrypt from 'bcryptjs'
import { getUserByEmail } from '@/lib/users'

const SECRET = 'your-secret-key'
```

```

export default async function handler(req, res) {
  if (req.method !== 'POST') return res.status(405).end()

  const { email, password } = req.body
  const user = getUserByEmail(email)

  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({ error: 'Invalid credentials' })
  }

  const token = jwt.sign({ email }, SECRET, { expiresIn: '1h' })
  res.status(200).json({ token })
}

```

7. Endpoint Terproteksi – pages/api/protected.js

```

import { authenticateToken } from '@/middleware/auth'

export default function handler(req, res) {
  const auth = authenticateToken(req, res)
  if (!auth.email) return

  res.status(200).json({ message: `Hello ${auth.email}, you are authenticated!` })
}

```

8. Pengujian API (contoh fetch dari frontend)

```

const login = async () => {
  const res = await fetch('/api/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email: 'user@mail.com', password: '123456' }),
  })

  const data = await res.json()
  return data.token
}

```

```
const getProtectedData = async (token) => {  
  const res = await fetch('/api/protected', {  
    headers: { Authorization: `Bearer ${token}` },  
  })  
  
  const data = await res.json()  
  console.log(data)  
}
```
